ResearchGate

See discussions, stats, and author profiles for this publication at: https://www.researchgate.net/publication/246115059

An algorithm for large scale 0–1 integer programming with application to airline crew scheduling

Article *in* Annals of Operations Research · December 1995

DOI: 10.1007/BF02099703 · Source: CiteSeer

CITATIONS
179

READS
1,634

1 author:

Dag Wedelin
Chalmers University of Technology
28 PUBLICATIONS **583** CITATIONS

SEE PROFILE

All content following this page was uploaded by Dag Wedelin on 15 September 2014.

The user has requested enhancement of the downloaded file.

An algorithm for large scale 0-1 integer programming with application to airline crew scheduling

Dag Wedelin

February 1993, revised December 1994

Abstract

We present an approximation algorithm for solving large 0-1 integer programming problems where A is 0-1 and where b is integer. The method can be viewed as a dual coordinate search for solving the LP-relaxation, reformulated as an unconstrained nonlinear problem, and an approximation scheme working together with this method. The approximation scheme works by adjusting the costs as little as possible so that the new problem has an integer solution. The degree of approximation is determined by a parameter, and for different levels of approximation the resulting algorithm can be interpreted in terms of linear programming, dynamic programming, and as a greedy algorithm. The algorithm is used in the CARMEN system for airline crew scheduling used by several major airlines, and we show that the algorithm performs well for large set covering problems, in comparison to the CPLEX system, in terms of both time and quality. We also present results on some well known difficult set covering problems that have appeared in the literature.

Contents

1 Introduction

In this paper we present an approximation algorithm for a particular class of 0-1 integer programming problems. It is designed to be efficient for very large problems, where other integer programming methods often fail. The algorithm is a development of the probabilistic method for combinatorial optimization in [19].

We consider the 0-1 integer programming problem (ILP)

$$\min cx$$

$$Ax = b$$

$$x_j \in \{0, 1\}$$

We will restrict our attention to the case where A is 0-1, and where b is integer. Also with these restrictions, the problem is NP-hard, and several well known NP-hard problems, such as the set partitioning, covering and packing problems, are conveniently stated in this way. The linear programming relaxation of (ILP) is (LP)

$$\min cx$$

$$Ax = b$$

$$0 \leq x \leq 1$$

Common solution methods for (ILP) are based on solving (LP), which can usually be done efficiently. If (LP) happens to give a 0-1 solution, this is also an optimal solution to (ILP), and for certain common and nontrivial subclasses of 0-1 problems, (LP) always has an integer solution. For more difficult problems particular instances may also be easy in this sense. If however the (LP) solution has many noninteger values, very little information about the solution to the 0-1 problem is obtained in this way, and typically techniques such as branch and bound have to be used to resolve the solution to integrality. This can work very well for small problems and also for larger problems with special structure, but nevertheless strongly limits the range and size of problems that can be solved. For a full presentation of existing methods, see for example [13, 16, 17].

Our algorithm takes another approach in that it does not consider the solution of (LP) as an intermediate step, but attempts to solve the 0-1 problem in a more direct way. As solutions we here consider feasible but possibly

non-optimal integer points, and if feasibility itself is non-trivial there is no guarantee that a solution will be found at all. It can be viewed as consisting of two interacting components: the first part is a simple dual search algorithm for finding a 0-1 solution to (ILP) by considering its lagrangian relaxation. The second part is an approximation scheme that manipulates the cost array c as little as possible, to enable convergence and to make the relaxed problem have a unique 0-1 solution. The magnitude of this manipulation is governed by a parameter κ. When $\kappa = 0$ the algorithm can only find optimal solutions, but often fails. When $\kappa = 1$ the algorithm works as a greedy algorithm that strives to converge quickly to any feasible solution. In practice it is difficult to know in advance what parameter value to choose, so the iteration starts with $\kappa = 0$, then slowly increasing the value until convergence occurrs. We note however that the algorithm usually does not require problem specific analysis or tuning to work well.

The paper is organized in the following way. In section 2 we present the algorithm itself. The following sections 3 to 5 analyze and interpret the algorithm for different values of the approximation parameter. The algorithm has been extensively tested on large set covering problems, arising in the field of airline crew scheduling, and in section 6 we briefly describe this application and present some comparative experimental results. We also present results on some well known difficult set covering problems that have appeared in the literature.

2 The algorithm

Consider the following *lagrangian relaxation* (R) of (ILP)

$$\min_{0 \leq x \leq 1} cx + y(b - Ax)$$

This is a relaxation for all y, since any x feasible for (ILP) is also feasible for (R), with the same objective value. As usual, the idea with the relaxation is to find some vector y so that the solution to the relaxed unconstrained problem (R) is feasible also for (ILP). Since both objectives are the same, this solution will then also be optimal for (ILP), and we have solved our problem.

Solving (R) is trivial, and by defining the *reduced cost* vector

$$\overline{c} = c - yA \tag{1}$$

we may write the solution to (R) as

$$x_j = \begin{cases} 0 & \text{if } \overline{c}_j > 0 \\ \text{any value in } [0,1] & \text{if } \overline{c}_j = 0 \\ 1 & \text{if } \overline{c}_j < 0 \end{cases} \tag{2}$$

If $\overline{c}$ is nonzero, the solution is integer and unique. However, if some $\overline{c}_j$ are zero, it will usually be difficult to find a solution feasible for (ILP). We therefore wish to find some y for which all elements of $\overline{c}$ are nonzero, and where the resulting unique integer solution to (R) is feasible for (ILP). Such a y may not exist however, and we will usually have to introduce an approximation to resolve this situation.

We now turn to describing the algorithm itself. In the design of the algorithm, we have taken care to use only very simple operations, so that the algorithm will run also for very large problems. We first describe the algorithm without approximation, and then introduce the approximation scheme in subsection 2.2.

2.1 The algorithm without approximation

When no approximation is present, we apply a simple coordinate search taking every element of y in turn and iteratively choosing y_i so that the solution x to (R) satisfies constraint i of $Ax = b$. To ensure feasibility of this operation, we will assume that all $a_{ij} \in \{0, 1\}$ and that b is integer. It is possible to allow $a_{ij} \in \{-1, 0, 1\}$, which is a straightforward generalization, but we choose to present the algorithm in a form for which we also have computational experience.

We now explain how y_i is computed. Since this calculation is repeated many times for all possible i it is useful to maintain and update also the current value of $\overline{c}$ during the computation. In the subsequent defintions we consistently use i to index any row-related entity and similarly j for columns. Define for each row i the array $\overline{c}^i$ as the elements $\overline{c}_j$ of $\overline{c}$ for which it holds that $a_{ij} = 1$ and let the array

$$r^i = \overline{c}^i + y_i \tag{3}$$

The purpose of (3) is to cancel from $\overline{c}$ the effect of any previous value for y_i. Let r^- be the b_i'th lowest element of r^i, and r^+ the $b_i + 1$'th lowest. The

new value of y_i is then chosen as

$$y_i := \frac{r^- + r^+}{2} \tag{4}$$

If $r^+$and r^- are nonzero the result after updating of $\overline{c}$ is that exactly b_i of the elements of $\overline{c}^i$ are negative. We note that all these computations are very simple and well suited for fast implementation. We also note that it in this case is possible to work with $\overline{c}^i$ directly, rather than r^i, and update y_i differentially.

For simple problems this algorithm alone may solve the problem, but commonly $\overline{c}$ converges to some point where many elements are 0, and no integer solution is found. The remedy is the approximation scheme, which provides fast convergence and also makes it possible to find approximate 0-1 solutions.

2.2 The approximation algorithm

The idea of the approximation scheme is to distort c as little as possible, allowing for an integer solution to be found by coordinate search. The magnitude of the distorsion is determined by the parameter κ, which therefore in some sense determines the degree of approximation. When $\kappa = 0$ there is no approximation at all, and for $\kappa = 1$ approximation is maximal. In this respect, the use of the parameter is similar to the use of the so called temperature parameter in simulated annealing methods(see [1]).

The distorsion is applied by modifying the coordinate search algorithm in the following way. Rather than having only one value y_i for each constraint, each constraint is associated with *two different* values y_i^- and y_i^+. The idea is to add a small positive value to elements of $\overline{c}^i$ that are positive after the iteration of y_i, and similarly, a negative value to the elements that are negative. The expressions for y_i^+ and y_i^- are

$$y_i^- = y_i + \frac{\kappa}{1-\kappa}(r^+ - r^-) + \delta \tag{5}$$

$$y_i^+ = y_i - \frac{\kappa}{1-\kappa}(r^+ - r^-) - \delta \tag{6}$$

which is to be compared with equation (4). The first term is the same as before, but there is also a difference term proportional to $r^+ - r^-$, the

magnitude of which is governed by κ. The expression $\frac{\kappa}{1-\kappa}$ is just an arbitrary mapping of the interval $[0,1]$ to $[0,\infty]$ and could be chosen in some other way. The small constant δ is needed to ensure that the elements of $\overline{c}$ are nonzero at all times.

When we have both y_i^- and y_i^+ it is necessary to modify the calculation of r^i so that the old values from the previous iteration of the constraint are correctly subtracted from each element. A convenient way of doing this is to define arrays s^i of the same length as r^i, where the offset is stored individually for each element of $\overline{c}^i$. The s^i are then remembered during the calculation, in addition to the array $\overline{c}$. The iteration of one constraint consists of the following steps:

1.
$$r^i := \overline{c}^i - s^i \tag{7}$$

2.
$$\begin{aligned}
r^- &:= \text{ the } b_i\text{'th lowest element of } r^i && (8)\\
r^+ &:= \text{ the } b_i+1\text{'th lowest element of } r^i && (9)\\
y_i^- &:= y_i + \frac{\kappa}{1-\kappa}(r^+ - r^-) + \delta && (10)\\
y_i^+ &:= y_i - \frac{\kappa}{1-\kappa}(r^+ - r^-) - \delta && (11)\\
s_j^i &:= \begin{cases} -y_i^- & \text{if } r_j^i \le r^- \\ -y_i^+ & \text{if } r_j^i \ge r^+ \end{cases} && (12)
\end{aligned}$$

3.
$$\overline{c}^i := r^i + s^i \tag{13}$$

The iteration proceeds by repeatedly cycling through the constraints, and for each constraint these steps are performed. We note that in principle, several constraints could be iterated in parallel, provided that they have no common variables.

In degenerate cases, it may happen that $r^- = r^+$, and some positions of s^i can be chosen in different ways. The array s^i should then be chosen indeterministically among those possibilities that satisfy constraint i, i.e. so that exactly b_i of the elements of $\overline{c}^i$ are negative.

2.3 How to choose κ and δ

The most critical factor of the approximation scheme is how to choose κ. Two empirical observations can be made. First, it is difficult to know in advance what is an appropriate value of κ, since different problems converge for different values. Secondly, results are usually better if the approximation level is as low as possible at convergence. One possible solution is then to begin with $\kappa = 0$, and let this value slowly increase during the iteration until convergence occurs. Unfortunately, this is too slow to be practical, so a modified scheme is applied.

First, a relatively fast sweep is performed, so as to give an approximate indication of the point of convergence. Then the algorithm is run again, with the change that in a relatively small interval below the old convergence point the sweep is very slow, to enable a more thorough search in this critical region. This procedure may be repeated in several trials for even better results. In this way, the number of iterations can usually be kept down to around 200 per trial, a figure which does not seem to be particularly sensitive to the size of the problem.

The value of δ should generally be chosen as large as possible without deteriorating the obtained solutions, to within the right order of magnitude. Commonly, a large value of δ lowers the point of convergence for κ and thus gives faster convergence, but there is no clear relationship between the quality of the solutions and δ. For a particular problem, it may often be possible to obtain better solutions, or to obtain better convergence, by testing some other δ. Other schemes for δ that depend on κ, or random offsets, can also be used, and may be preferred in certain cases, but no major performance differences have been found.

2.4 Inequality constraints

Since we are restricted to binary variables, slack variables for inequality constraints have to be introduced in a special way. For example, a constraint of the type

$$x_1 + x_2 + x_3 \leq 2$$

can be replaced by

$$x_1 + x_2 + x_3 + x_4 + x_5 = 2$$

by introducing the slack variables x_4 and x_5 at zero cost. Similarly, the constraint

$$x_1 + x_2 + x_3 \geq 2$$

can be replaced by

$$x_1 + x_2 + x_3 + x_4 + x_5 + x_6 = 5$$

With a proper implementation, the potentially large number of slack variables need not be represented explicitly, so there is no loss in efficiency when inequalities are used.

3 Linear programming interpretation

Noting that (R) is also a relaxation of (LP), we may for the case of no approximation, understand the algorithm entirely within the familiar framework of linear programming. We will first consider the coordinate search from a dual point of view, and then in section 3.2 give a theorem about the existence of relaxations with nonzero elements of $\overline{c}$.

3.1 Dual interpretation

It is well known that (LP) and its relaxation (R) is related to a nonlinear unconstrained dual problem of the form

$$\max_y f(y) \tag{14}$$

where $f(y)$ is the minimum of (R) as a function of y,

$$f(y) = yb + \min_{0 \leq x \leq 1} (c - yA)x \tag{15}$$

This problem is the dual of (LP) with respect to the constraints $Ax = b$ only, in contrast to the usually considered dual problem where all constraints except $x \geq 0$ are relaxed. We note that $f(y)$ can be viewed as the minimum of a set of linear functions in y, one for each instantiation of x, and that $f(y)$ is therefore piecewise linear and concave.

We now show that the coordinate search method in section 2.1 corresponds exactly to the problem of maximizing $f(y)$. For every y for which the partial

derivatives exist, they can be written

$$\frac{\partial f}{\partial y_i} = b_i - a_i x^R \tag{16}$$

where x^R is the solution to (R) for this y. We recall that in the coordinate search y_i is always chosen so that the solution to (R) satisfies $a_i x = b_i$, which gives us $\frac{\partial f}{\partial y_i} = 0$ for this point. Since $f(y)$ is concave we may therefore conclude that this point is also its maximum along y_i. We observe that if all $\overline{c}_j$ are strictly nonzero the maximum of $f(y)$ occurs on a plateau, and in n dimensions within a polytope with a positive volume, allowing a slack in all of the y_i. We also note that the coordinate search algorithm always chooses the value in the middle of this plateau, along the considered axis.

In figure 1*a* we show $f(y)$ for the problem

$$\min 6x_1 + 9x_2 + 2x_3 + 3x_4 + 5x_5 + 2x_6 + 5x_7$$

subject to the constraints

$$\begin{aligned} x_1 + x_2 + x_3 + x_4 + x_5 &= 2 \\ x_1 + x_2 + x_6 + x_7 &= 3 \end{aligned}$$

Since we have only two constraints, $f(y)$ becomes a function of two variables. Note the plateau at the top, this is where the solution to (R) satisfies the constraints. Figure 1*b* represents an aerial view of $f(y)$, in which the lines $c - yA = 0$ are plotted, and in each area the value of Ax is written. Note that the position of these lines are independent of b, while the value of $f(y)$ itself is not. From any starting point iteration interchangeably proceeds vertically and horizontally until the area 2,3 is reached. The horizontal iteration moves to an area where the first index is 2, and the vertical moves to an area where the second index is 3.

A common problem with a repeated line search along fixed directions on a function whose first derivatives are not continuous, is that the procedure may get stuck over a ridge where none of the coordinate directions give an increase in $f(y)$, but where some other direction will. This is what happens when $\overline{c}$ is cluttered with zeros, so that for some constraints $r^- = r^+$, and this method would in most cases be useless without introducing the approximation. Also, convergence may be slow. An example of a situation where convergence is a problem is shown in figure 2. The dark area indicates the area where $f(y)$

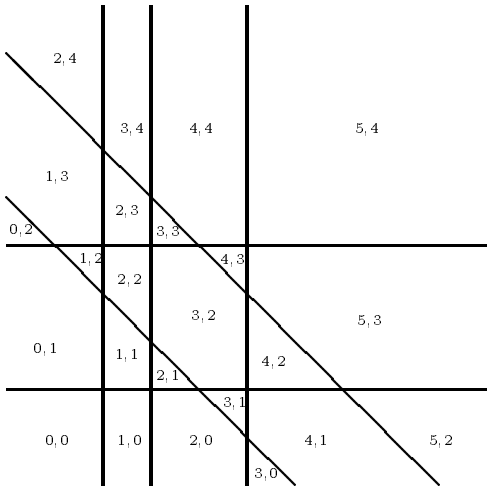

Figure 1: *a)* Plot of $f(y)$. *b)* The constraints in dual space.

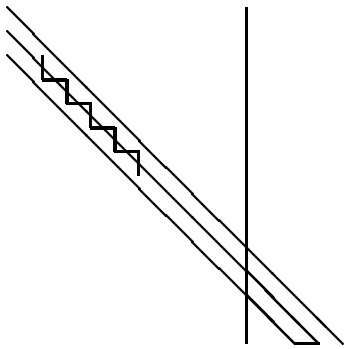

Figure 2: Slow convergence.

reaches its maximum. We see that the speed of convergence depends on the distance between the three parallel lines, and if these were to be placed on top of each other the procedure would be stuck. Note that the dark area would then be reduced to a line, so that no unique 0-1 solution would exist.

By using y_i^- and y_i^+ the slow convergence illustrated in figure 2 is eliminated, since these offsets effectively move the planes apart. Note that the restrictions on A and b ensure that along each coordinate, the maximum of $f(x)$ always lies between two planes, and never on a single plane, so that this is always possible. We therefore obtain improved convergence, at the price of approximation. In practice, convergence in itself does not seem to be a problem, even for very large problems. The difficulty is to approximate as little as possible, to obtain good integer solutions.

3.2 Existence of relaxation with $\bar{c} \neq 0$

We will now consider the existence of a relaxed problem with nonzero reduced costs. It is then useful to consider the dual of (LP), where also the constraints $x \leq 1$ are relaxed. We have (DLP)

$$\max_{y,y^s} yb + y^s \, 1^n$$

$$yA + y^s \leq c$$

$$y^s \leq 0$$

where we let 1^n be an array of length n with ones, and where we denote the dual variables for $Ax = b$ by y and the dual variables for $x \leq 1$ by y^s. For any value of y we can make the solution dual feasible by choosing y^s appropriately. In order to maximize the objective function $yb + y^s\, 1^n$, y^s should for any given y be chosen as

$$y^s(y) = \min(c - yA, 0^n) \tag{17}$$

where the min-function operates elementwise on the arguments. We may then view the objective function as a function of y only

$$f(y) = yb + \min(c - yA, 0^n)\, 1^n \tag{18}$$

giving us the same function $f(y)$ as before. Maximization of $f(y)$ can thus be viewed as a way of solving the dual problem (DLP) by moving within the dual feasible set.

Theorem 1 *A y such that (R) gives a unique solution to (LP) exists if and only if (LP) has a unique solution that is 0-1.*

We note that when the condition of the theorem is satisfied, (LP) itself gives an integer solution, which is the traditional easy case. We also note that the theorem is true for arbitrary A and b, without restriction.

In the proof we will use the notion of *strong complementary slackness*, which can be stated as follows(see [13]): if the primal and dual problems have optimal solutions there exists at least one pair of optimal solutions x and y such that $c_j - ya_j = 0$ exactly when $x_j > 0$.

Proof We have already observed that if (R) has a unique solution, $f(y)$ has well defined partial derivatives that are all 0, and that it must be optimal. To prove that x must also be the only solution, let $\Delta b = Ax - b$, and consider how the quantity $d\Delta b$ changes as we move in some direction d from the polytope where $\Delta b = 0^m$. A change occurs only when we pass some plane $c_j - ya_j = 0$, and the change in Δb is a_j if the scalar product da_j is positive and $-a_j$ if da_j is negative. We see that $d\Delta b$ is therefore always positive, as soon as we have moved outside at least one plane $c_j - ya_j = 0$. Since all points in space can be reached by choosing the appropriate direction d, we conclude that no other polytope can satisfy $Ax - b$, and the solution must therefore be unique.

Conversely, assume that x is a unique optimal 0-1 solution to (LP) and (y, y^s) is an optimal solution to (DLP). Since $y^s \leq 0$, it follows from the theorem of strong complementary slackness that it is possible to choose y so that $\overline{c}_j \leq 0$ whenever $x_j = 1$ and $\overline{c}_j > 0$ otherwise. At this point $ya_j > 0$ whenever $\overline{c}_j = 0$ since otherwise it would be possible to increase $yb + y^s 1^n$ by replacing y by $y' = (1 + \delta)y$ for some small δ, contradicting that y is a solution to the dual. We can therefore always choose a $\delta > 0$ so that $c - y'A > 0$ for $x = 1$ and $c - y'A < 0$ otherwise.□

4 Dynamic programming interpretation

We will show that for $\kappa = 1/3$ we may interpret our algorithm as an algorithm for nonserial dynamic programming (see [5]). The dynamic programming algorithm is in some respects more general and will first be described in detail.

Consider minimizing the *separable* function

$$\min c(x) = \sum_i q(x^i) \tag{19}$$

where the $q(x^i)$ are arbitrary functions over subsets of x. We will consider the case where the x_j are binary, but the algorithm works for any finite domain. This problem is NP-hard in general, but can be efficiently solved if the hypergraph determined by the sets x^i is acyclic. Let for this hypergraph, j index the vertices and i index the hyperedges defined by the index sets E_i of x^i. In this case, every vertex can be viewed as the root of a hypertree. Let T_{ij} be the subset of all hyperedges in the subtree defined by vertex j and edge i, edge i included. Also let N_j be the set of edges connected to vertex j.

Define the *marginal cost* function for variable x_j as

$$c(x_j) = \min_{x \setminus x_j} c(x) \tag{20}$$

If all $c(x_j)$ were known, the main problem would be solved by the trivial minimizations $\min c(x_j)$ for every j. Assuming acyclicity, we will now develop an efficient algorithm for computing these functions.

Let $s_i(x_j)$ be the minimum over the potentials in subtree T_{ij} only, as a function of x_j,

$$s_i(x_j) = \min_{x \setminus x_j} \sum_{i \in T_{ij}} q(x^i) \tag{21}$$

We may write $c(x_j)$ as the sum of the minimizations for each subtree,

$$c(x_j) = \sum_{i \in N_j} s_i(x_j) \tag{22}$$

since these subproblems have only variable x_j in common. The $s_i(x_j)$ may in turn be written recursively as

$$s_i(x_j) = \min_{x^i \setminus x_j} q(x^i) + \sum_{l \in E_i \setminus j} r_i(x_l) \tag{23}$$

where $r_i(x_j)$ is the sum of all $s_k(x_j)$ for all $k \in N_j \setminus i$, or equivalently

$$r_i(x_j) = c(x_j) - s_i(x_j) \tag{24}$$

We will now develop an iterative algorithm for computing all $c(x_j)$ using the given equations. It is convenient to visualize these computations as a tree structured network, where the $c(x_j)$ are consistently maintained in the nodes, the $r_i(x_j)$ are input to arcs, the $s_i(x_j)$ are output, and the calulations (23) are performed in each arc. For efficiency, the computation in an arc is organized so that all outputs $s_i(x_j)$ from arc j are computed together. An arc update then consists of the following steps, which may be compared with the steps of our approximation algorithm in section 2.2:

1. For all $j \in E_i$:
$$r_i(x_j) := c(x_j) - s_i(x_j) \tag{25}$$

2. For all $j \in E_i$:
$$s_i(x_j) := \min_{x^i \setminus x_j} q(x^i) + \sum_{l \in E_i \setminus j} r_i(x_l) \tag{26}$$

3. For all $j \in E_i$:
$$c(x_j) := r_i(x_j) + s_i(x_j) \tag{27}$$

We observe that since the output from an arc $s_i(x_j)$ is recursively dependent only on other $s_i(x_j)$, convergence will occur after d iterations of all arcs, where d is the *diameter* of the tree, i.e. the length of the longest path between two nodes. An important point is that by organizing the calculations in this particular way, all computations are local in the network and can be carried out even if the structure is not acyclic, this is how approximation can be introduced in this interpretation.

We will now demonstrate the correspondence between the two algorithms, and begin by specializing the DP algorithm to the binary case. Assuming binary x_j, define

$$C_j = c(x_j = 1) - c(x_j = 0) \tag{28}$$

We note that the differences C_j are all that is needed to solve the simple optimization problems for each variable. Define R_{ij} and S_{ij} similarly, so that the relation

$$C_j = R_{ij} + S_{ij} \tag{29}$$

holds, and can be used to modify equations (25) and (27). Using the new notation, we may now replace (26) by the calculation

$$S_{ij} = s_i(x_j = 1) - s_i(x_j = 0) \tag{30}$$

where

$$s_i(x_j) = \min_{x^i \setminus x_j} q(x^i) + \sum_{l \in E_i \setminus j} R_{il} x_l + K \tag{31}$$

In this expression, K is some unknown constant that is however not a function of x_j, so it is cancelled out when S_{ij} is computed.

We now turn to how (ILP) can be modelled as the problem (19). For every constraint, define a function $q(x^i)$ which is 0 if constraint $a_i x = b_i$ and ∞ otherwise. Also, for each term $c_i x_i$ in the objective cx we add a function $q(x_i)$ with $q(x_i = 1) = c_i$ and $q(x_i = 0) = 0$. Note that for full correspondence with the dynamic programming interpretation we must assume that the constraint structure of (ILP) is acyclic.

If the constraint defined by $q(x^i)$ is complicated, there is no other way to solve this problem short of testing all the configurations of x^i, which is exponential in the number of variables active in the constraint. For constraints with a simple structure however, it is may be possible to obtain solutions more efficiently. The linear constraint $ax = b$ where a is binary and b is integer, is such a case. The minimization in (31) then becomes a simple

instance of the knapsack problem where one just chooses the variables in order of increasing R_{ij}. Note that this problem would be NP-hard if a was arbitrary. Let r^- be the b'th lowest value among the R_{il} considered for some particular i, and let r^+ be the $b+1$'th lowest. We then obtain

$$S_{ij} == \begin{cases} -r^- & \text{if } R_{ij} \geq r^+ \\ -r^+ & \text{if } R_{ij} \leq r^- \end{cases} \tag{32}$$

The identity between the two algorithms is now established for $\kappa = 1/3$ by identifying the elements of $\overline{c}$ with the values C_j, the elements of r^i with the R_{ij} and the elements of s^i with the S_{ij}. We note that if the constraint structure of (ILP) is not acyclic, the dynamic programming interpretation is no longer valid, but the correspondence between the iterative algorithms remains. We conclude by stating the finite convergence property of the DP algoritm also for the 0-1 programming algorithm:

Theorem 2 *If $\kappa = 1/3$ and the constraint structure is acyclic, the algorithm converges to the optimal solution in d steps, where d is the diameter of the hypergraph defined by the constraints.*

5 Greedy algorithm interpretation

When $\kappa = 1$ the second terms of (5) and (6) will dominate and set each value of $\overline{c}^i$ to $-\infty$ or ∞. Satisfaction of the constraints then completely overrides the cost structure of the problem. The algorithm then behaves as the following simple greedy algorithm.

Let each variable have three possible values: 0, 1 or unknown. Initially all variables are set to unknown. Repeatedly pass through the constraints and assign 0 or 1 to each of the unknown constraint variables, so that the current constraint is satisfied. The assignment is based on the values of c, so that the variables are set to 1 in order of increasing cost. Variables that are already set to 0 or 1 are not allowed to change, except when the unknown variables are insufficient for satisfying the constraint, in which case an old assignment may be overridden.

Generally, this algorithm may continue to oscillate forever, since it may fail in finding a solution, or because no solution exists. However, in the frequent special case when the equalities are replaced systematically by $\geq$

or $\leq$, convergence is guaranteed after exactly one pass over the constraints. This is because old assignments are overridden in one direction only, from 0 to 1 for $\geq$ and from 1 to 0 for $\leq$. The result is that a constraint is never violated after it has been passed, and a suboptimal solution is found very quickly.

6 Computational results

The algorithm has been extensively tested on large set covering problems arising from airline crew scheduling applications. It is included in the CARMEN system for airline crew scheduling, and is used by the airlines SAS, Lufthansa, Alitalia and KLM. Presently, this is the most important application for which the algorithm has found serious use, and we will therefore give a short introduction.

6.1 The airline crew scheduling problem

This problem has been extensively studied for many years (see [2, 9]), and we will describe the problem in its classical and basic form. Assume that a timetable with all flight legs is given (a flight leg is a given flight at a given time). To all these flight legs we wish to assign crews so that every crew receives a reasonable schedule, and so that the cost of the solution is as small as possible. Assignment of individuals to crews is performed separately, and is not considered here.

Every crew has a home base, and the schedule for a crew is described as a sequence of flight legs starting and ending at the home base. Such a sequence is called a pairing. A pairing may range over one or several days. A legal pairing must satisfy a large number of legal and contractual rules, such as the maximum flight time per day, minimum time for rest and so on. While these rules are difficult to define mathematically, it is quite possible to generate pairings which satisfy the rules. The problem is therefore commonly solved in two steps, which are repeated over and over again:

1. Generate a large number of legal pairings and compute the cost for each pairing, using any previously found solution as a guideline.
2. Select a subset of the generated pairings so that every flight in the

schedule is included in *at least one* of the selected pairings and in such a way that the total cost is minimized. The total cost is calculated as the sum of the costs for the individual pairings.

Every flight leg must be included in at least one pairing since there must be at least one crew for every flight. However, we allow that another crew may travel on a flight as passengers, if this gives a lower total cost.

The second step can be expressed as solving a problem of the form (SC)

$$\min cx$$

$$Ax \geq 1$$

$$x_j \in \{0, 1\}$$

where A is a 0-1 matrix. This problem is known as the set covering problem, and is NP-hard. In our case, each row of A corresponds to a flight leg and each column to a pairing. Any extra cost for overcovers can be taken account for within the set covering formalism by changing the cost function appropriately. A typical problem deals with about 50 to 2000 flights and 10000 to 1000000 pairings. The normal number of nonzero elements of A is between 5 to 10 per column.

Due to the complexity of the application it may be desirable to extend the problem in various ways. One such extension is the addition of so called base constraints, and in the CARMEN system, the algorithm is implemented in a version that can also handle these constraints.

In modern approaches to the airline crew scheduling problem there is often feedback of dual variables from the set covering LP-relaxation to the pairing generator (see[8, 15]), to allow for a very specific generation of pairings that can improve the present solution. We note that our algorithm can be used to provide similar output.

6.2 Test results

The algorithm has been tested against the CPLEX-system for linear and integer optimization (version 2.1beta), see [7]. CPLEX is an optimization system that is used by several major airlines specifically for the type of problems that are considered here. The size of the problems before and

	size before preprocessing			size after preprocessing		
problem	rows	columns	nonzero	rows	columns	nonzero
B727scratch	254	282	2675	29	157	372
ALITALIA	1255	3438	41093	118	1165	4155
A320	1255	9678	125156	199	6931	31801
A320coc	1224	43492	546474	235	18753	82229
SASjump	3482	15485	121902	742	10370	42695
SASD9imp2	4116	58850	301614	1366	25032	110881
SASD9imp1	4089	155660	982751	1585	105804	566905

Table 1: Size of problems before and after preprocessing.

	CPLEX				our method	
problem	obj	time	LP	time LP	obj	time
B727scratch	92800	0.2s	92450	0.07s	92800	1.4s
ALITALIA	5017500	1s	5017500	0.7s	5017500	11s
A320	529250	23s	529250	17s	529250	1m4s
A320coc	565000	2m12s	564550	1m40s	565000	5m2s
SASjump	4737768	23m	4734156	7m40s	4737892	3m58s
SASD9imp2	4335780	5h53m	4332348	16m	4333450	7m46s
SASD9imp1	-	-	4329260	3h30m	4329750	36m10s

Table 2: Test results for real set covering problems.

after preprocessing is shown in table 1. In the preprocessing stage as much as possible of the problem is eliminated by simple criteria. We see that for some problems we have a very significant reduction, but for problems with many rows the reduction is more moderate. The remaining "difficult" part of the problem is then solved by an optimizing algorithm and the results are summarized in table 2. The times indicated are total running times for an HP720 workstation, including the time for I/O operations.

In addition to the integer solution, the CPLEX system provides information about the objective value for the LP relaxation and for the first four problems it can also prove that the solution is optimal. We see that for the tested

problems the difference between the 0-1 and LP objective is very small. The parameter settings have been recommended by CPLEX specifically for use with these problems.

The times for our algorithm are for 5 trials. The total execution time is essentially proportional to the number of trials, so it is possible to receive solutions considerably faster if fewer trials are specified. All results have been generated with standard parameter settings.

For practical purposes both algorithms give solutions of about the same quality for these problems. By tuning of the parameters individually for each problem both CPLEX and our method can receive marginally better results on SASjump and SASd9imp2, but the here presented results reflect the true behaviour in a production environment. We note however that the running time for CPLEX increases dramatically with the number of constraints in the problem, and that it fails to solve the largest problem in the test set. For our algorithm, the running times are consistently proportional to the size of the problem, and we can without difficulty solve considerably larger problems than those used in this comparison. We also note that for the largest problem, the total running time is considerably lower than the time required by CPLEX to find an LP solution.

We have also tested the algorithms on randomly generated problems, for which the difference between the LP and ILP objective is larger. On these problems preprocessing hardly makes any difference, and virtually no variables in the LP solution have integer values. With our algorithm, and with the same standard parameter settings, we quickly obtain 0-1 solutions that are considerably better than those of any greedy algorithm. No interesting comparison was possible however, since the CPLEX system is not at all suited for such problems, and was only able to find extremely bad solutions.

Considering memory, our implementation has the same memory requirements during the whole run, while CPLEX allocates more memory during the solution process. On a given computer, it is therefore possible to solve problems about 5 times as large with our algorithm than with the version of CPLEX that we had available. The approximate limit for this type of problems with 128 MB of memory is about 1000000 columns for our algorithm and 200000 for CPLEX.

As a final example we consider a type of set covering problems arising from Steiner triple systems, that have appeared in the literature as especially

	size		our method			old results	
problem	rows	columns	trials	time	obj	old obj	optimal
A27	116	27	5	12s	18	18	yes
A45	330	45	5	40s	31		
			45	6m22s	30	30	yes
A81	1080	81	5	2m23s	61	61	unknown
A135	3015	135	5	6m50s	106		
			13	18m17s	104	105	unknown
A243	9081	243	5	25m42s	205		
			30	2h32m	203	204	unknown

Table 3: Test results for difficult set covering problems.

difficult problems suitable for test purposes, see [11, 3, 10, 14]. These problems, which can be generated recursively by a simple rule (see [14]), have a completely different structure compared to the airline problems. They have very few variables and a large number of constraints, where every constraint contains exactly three variables. We present our results in table 3. These problems have been run on a Macintosh 6100, and the previously known best results have been taken from the most recent paper [14]. We see that all problems are solved at least to their previously best values, and for the two largest problems slightly better solutions were found. We also see that more than 5 trials were necessary to solve all problems to this level, which resulted in a corresponding increase of the execution time. We emphasize however that no special tuning was performed and that the same program with the same standard parameter settings as for the airline problems was used. Compared to the method of [14] our algorithm seems to be considerably faster, although it in turn seems to be considerably slower than the specialized heuristic of [10].

7 Conclusion

We have presented an approximation algorithm for solving 0-1 integer programming problems. The computations are based on very simple operations that can be organized in a network having the same topology as the con-

straint structure. A parameter determines the compromise between the running time and the quality of the solution. When $\kappa = 0$ we have shown that the algorithm has a close relationship to the corresponding LP-problem. At $\kappa = 1/3$ the algorithm can be interpreted in terms of dynamic programming, and when $\kappa = 1$ it turns into a greedy algorithm.

We have shown that the algorithm is able to obtain high quality solutions to large set covering problems. The tests indicate that it compares well with established methods based on linear programming, especially when the problems are combinatorially difficult. We have also shown that the method works well in a production system for airline crew scheduling.

One obvious direction for further development is to better understand the matematical properties of the approximation scheme, and to further develop the relationship between this algorithm, dynamic programming, and linear programming. Developments of the current algorithm could be to find a more efficient sweep mechanism, and to eliminate the restrictions on A and b. Another possibility could be to generalize the approximation scheme, and make it independent of the coordinate search, allowing it to work together with other linear programming methods, although this would probably invalidate the network interpretation.

Acknowledgements

I am deeply indebted to Erik Andersson, who has inspired the development of this algorithm, and who has also carried out several of the tests in section 5. I also wish to thank Tony Elmroth for detailed comments on the performance of the algorithm.

References

[1] Aarts E.H.L. and Korst J.H.M. (1989). Simulated Annealing and Boltzmann Machines. Wiley.

[2] Arabeyre J.P., Fearnley J., Steiger F.C. and Teather W. (1969) The Airline Crew Scheduling Problem: A Survey. Transportation Science 3, 140-163.

[3] Avis D., (1980) A note on some computationally difficult set covering problems. Mathematical Programming 18, 138-145.

[4] Balas E. and Ho A. (1980). Set Covering Algorithms using Cutting Planes, Heuristics and Subgradient optimization: a Computational Study. Mathematical Programming Study 12, 37-60.

[5] Bertele U. and Brioschi F. (1972). Nonserial Dynamic Programming. Academic Press.

[6] Chvatal. V (1979). A greedy-heuristic for the Set-Covering problem. Math. Oper. Res. 4, 233-235.

[7] CPLEX Reference Manual (1992). CPLEX Optimization Inc.

[8] Derosiers J., Dumas. Y, Solomon M.M. and Soumis F. (1993). Time Constrained Routing and Scheduling. To be published in Handbooks in Operations Research and Management Science, volume on networks. North-Holland.

[9] Etschmaier M.M. and Mathaisel D.F. (1985). Airline Scheduling: an overview. Transportation Science 19,127-138.

[10] Feo T.A. and Resende M.G.C. (1989). A probabilistic heuristic for a computationally difficult set covering problem. Operations Research Letters 8, 67-71.

[11] Fulkerson D.R., Nemhauser G.L., Trotter Jr L.E. (1974). Two computationally difficult set covering problems that arise in computing the 1-width of incidence matrices of Steiner triple systems. Mathematical Programming Study, 72-81.

[12] Garey M.R and Johnson D.S. (1979). Computers and Intractability: A Guide to the Theory of NP-Completeness. W.H. Freeman & Co.

[13] Hu T.C. (1969). Integer Programming and Network Flows. Addison-Wesley.

[14] Karmarkar N., Resende M.G.C. and Ramakrishnan K.G. (1991). An interior point algorithm to solve computationally difficult set covering problems. Mathematical Programming 52, 597-618.

[15] Lavoie S., Minoux M. and Odier E. (1988). A new approach for crew pairing problems with an application to air transportation. European Journal of Operations Research 35, 45-58.

[16] Nemhauser G.L. and Wolsey L.A. (1988). Integer and Combinatorial Optimization. Wiley.

[17] Schrijver A. (1986). Theory of Linear and Integer Programming. Wiley.

[18] Syslo M., Deo N. and Kowalik J.S. (1983). Discrete Optimization Algorithms. Prentice-Hall.

[19] Wedelin D. (1989). Probabilistic Networks And Combinatorial Optimization. Technical report 49, Dept. of Computer Science, Chalmers Univ. of Tech.

[20] Wedelin D. (1993). Probabilistic Inference, Combinatorial Optimization and the Discovery of Causal Structure from Data. PhD Thesis. Dept. of Computing Science, Chalmers Univ. of Tech.

Theorem 3 *If A is 0-1, b is integer, and (LP) has a unique 0-1 solution, every limit point of the coordinate ascent method is a maximum point of* $f(y)$.

proof In a limit point, it must hold for each coordinate direction that either both the left and right derivatives are 0, or else that the left derivative is positive and the right derivative is negative. We assume wlg. that this point is the origin. We will prove that if not all derivatives are 0 there can be no polytope in dual space with a positive volume satisfying $Ax = b$. First assume that in all coordinate directions, the left derivative is positive and the right derivative is negative. Choose some coordinate yi, and choose some direction d for which $d_i = 0$, along which we choose the other coordinates. We will now consider the planes $c - yA = 0$ as a function of y_i and $Y_i = \sum ky_k - y_i$. In figure **??** we show an example of what this might look like. We observe that in this diagram, all lines must have a derivative between 0 and -1. If y_i is positive, we realize that

□

View publication stats